MapMind – Computer Vision Pipeline

1. Overview

MapMind converts a single indoor floor■plan image into a navigable graph (rooms ↔ doors ↔ corridors) using only classical computer■vision techniques and Python + OpenCV. The entire pipeline runs on CPU and is fully covered by 117 passing unit / integration tests.

2. Pipeline Steps

2.1 Binarisation & text■artifact suppression

> The image is gray■scaled and adaptively thresholded (colour■saturation ratio 0.015, achromatic delta 28). Isolated label■like components are removed with `_suppress_text_artifacts`, preserving narrow door jambs.

2.2 Morphological closing & adaptive gap selection

> A binary closing bridges small wall gaps. Six gap■ratio candidates (0.03■0.09) are evaluated; the ratio that maximises the number of valid rooms + corridors is selected.

2.3 Wall detection & merging

> Canny edge detection → Hough line transform. Line segments are merged if the angle ≤ 7.5° and endpoints are within 0.012 × image size.

2.4 Region segmentation

> Free■space mask (inverted walls) is split into connected components. Corridors are identified by aspect ≥ 4, short■side ≤ 0.16 × max width and solidity ≥ 0.20; the rest are rooms.

2.5 Door candidates

> Two families: (i) symbolic door blobs (colour■based), (ii) region■gap zones between region masks. Both are deduplicated (centres within 5 % of image size).

2.6 Probe■along■orientation

> From the door centre, rays are cast horizontally and vertically up to ≈10 % of image size. The first region label encountered on each side is recorded. If two distinct labels are found, the door is accepted; the orientation yielding the smallest combined centre■to■door distance is chosen (the ‘nearest■door’ fix).

2.6 Navigation■graph construction

> networkx.Graph: nodes = room/corridor names; edges = accepted doors; edge weight = centre→door + door→centre distance. Isolated nodes (e.g. Room_26) are recorded.

3. Configuration (CVConfig)

Parameter	Value
max_dimension	1200■px – working■resolution longest side
adaptive_segmentation	True – loop over gap■ratio candidates
segmentation_gap_candidates	[0.03, 0.04, 0.05, 0.06, 0.075, 0.09]
door_merge_distance_ratio	0.055■×■image size
door_probe_step_ratio	0.0075

door_probe_max_ratio	0.10
door_association_margin_ratio	0.01
door_fallback_distance_ratio	0.08
text_artifact_min_side_ratio	0.006
text_artifact_max_side_ratio	0.08
text_artifact_min_fill_ratio	0.25
text_artifact_max_fill_ratio	0.85
text_artifact_max_area_ratio	0.0015
wall_min_length_ratio	0.02
hough_threshold_ratio	0.035
hough_max_gap_ratio	0.0125
merge_angle_deg	7.5
merge_distance_ratio	0.012
merge_gap_ratio	0.012
wall_support_min	0.52
wall_support_radius	1
room_min_area_ratio	0.004
room_min_dim_ratio	0.022
room_max_area_ratio	0.8
corridor_aspect_ratio	4.0
corridor_max_width_ratio	0.16
door_merge_distance_ratio	0.055
door_duplicate_pair_ratio	0.14
door_min_area_ratio	0.00008
door_max_area_ratio	0.006
door_region_gap_min_area_ratio	0.00025
door_region_gap_max_major_ratio	0.12
door_min_dim_ratio	0.0075
door_max_minor_ratio	0.08
door_probe_step_ratio	0.0075
door_probe_max_ratio	0.1
door_association_margin_ratio	0.01
door_fallback_distance_ratio	0.08
ocr_min_confidence	35.0
debug	True – saves debug images

4. Test Results

The test suite contains 117 unit / integration tests. All pass (117 / 117). Tests verify door■pair uniqueness, graph consistency, correct overlay shape, coordinate round■trip, adaptive gap selection, and removal of text artefacts.

5. Limitations

No OCR back■end installed → all rooms/corridors receive generic names (Room_N / Corridor_N).

Isolated regions may appear (e.g., Room_26 in the apartment) when a door is not detected.

Graph connectivity depends on correct wall / door detection; complex or non■standard floor■plan conventions can break the pipeline.

The NLQ interface currently supports only a handful of pre■programmed intents; free■text queries produce a clarification prompt.

The system runs on CPU only; very small doors may be lost during downscaling beyond the working resolution (max_dimension = 1200 px).